

# Data Loading (ETL) Assignment

Dataset:

| Order_ID | Customer_ID | Sales_Amount   | Order_Date |
|----------|-------------|----------------|------------|
| O101     | C001        | 4500           | 12-01-2024 |
| O102     | C002        | Null           | 15-01-2024 |
| O103     | C003        | 3200           | 2024/01/18 |
| O101     | C001        | 4500           | 12-01-2024 |
| O104     | C004        | Three Thousand | 20-01-2024 |
| O105     | C005        | 5100           | 25-01-2024 |

Q1. Data Understanding

Identify all data quality issues present in the dataset that can cause problems during data loading.

Answer:

During the data loading phase of an ETL pipeline, the destination system expects data to be clean, consistent, and structurally valid. Analyzing the given dataset reveals four distinct data quality issues that would either cause the loading process to fail or silently corrupt the target table.

**Issue 1: Duplicate Record (Order\_ID O101)** Order\_ID O101 for Customer C001 with Sales\_Amount 4500 and Order\_Date 12-01-2024 appears twice in the dataset. This is an exact duplicate across all four fields. During loading, this would inflate sales totals and order counts, directly violating primary key constraints if Order\_ID is set as a unique key in the destination table, potentially causing the load job to fail or insert redundant data.

**Issue 2: Missing Value in Sales\_Amount (Order\_ID O102)** The Sales\_Amount field for Order O102 is Null. Since Sales\_Amount is a critical numeric field used in revenue calculations and aggregations, a null value here would either be rejected by a NOT NULL constraint at the destination or silently produce incorrect totals in downstream dashboards and reports.

**Issue 3: Inconsistent Date Format (Order\_ID O103)** The Order\_Date for O103 is recorded as 2024/01/18, while all other records follow the DD-MM-YYYY format. This inconsistency means the loading system cannot reliably parse the date column as a single uniform type. Depending on the destination schema, this record may either throw a type mismatch error or be loaded incorrectly, causing wrong date-based filtering and reporting.

**Issue 4: Invalid Data Type in Sales\_Amount (Order\_ID O104)** The Sales\_Amount for Order O104 is entered as "Three Thousand", which is a text string in a field that should contain a numeric value. This is a data type violation. When the ETL loader attempts to insert this value into a numeric or integer column in the destination database, it will raise a type conversion error and either reject the entire batch or skip the record, depending on the error handling configuration.

## Q2. Primary Key Validation

Assume Order\_ID is the Primary Key.

- a) Is the dataset violating the Primary Key rule?
- b) Which record(s) cause this violation?

### Answer:

**Primary Key Rule:** Every value in the Primary Key column must be unique and not null. No two rows can share the same Primary Key value.

#### **a) Is the Dataset Violating the Primary Key Rule?**

**Yes.** The dataset clearly violates the Primary Key rule because Order\_ID O101 appears more than once in the table, which directly breaks the uniqueness constraint that a Primary Key enforces.

#### **b) Which Records Cause This Violation?**

| Row   | Order_ID | Customer_ID | Sales_Amount | Order_Date |
|-------|----------|-------------|--------------|------------|
| Row 1 | O101     | C001        | 4500         | 12-01-2024 |
| Row 4 | O101     | C001        | 4500         | 12-01-2024 |

Both rows are completely identical across all fields, making this an exact duplicate and a direct Primary Key violation.

### **Impact of This Violation During Loading**

If the destination database has a **PRIMARY KEY** or **UNIQUE constraint** defined on Order\_ID, the ETL loader will throw a duplicate key error when it attempts to insert the second O101 record, and the load job may fail entirely depending on error handling configuration. Even if no constraint exists at the database level, the duplicate will silently corrupt aggregations, order counts, and revenue totals in all downstream reports.

### **Resolution**

The duplicate record must be identified and removed during the transformation stage using deduplication logic before the data reaches the loading phase. A simple approach would be to apply a distinct check or row-number partitioning on Order\_ID and retain only the first occurrence.

### Q3. Missing Value Analysis

Which column(s) contain missing values?

a) List the affected records

b) Explain why loading these records without handling missing values is risky

#### Answer:

Only the **Sales\_Amount** column contains a missing value in this dataset.

#### a) Affected Records

| Order_ID | Customer_ID | Sales_Amount | Order_Date |
|----------|-------------|--------------|------------|
| O102     | C002        | Null         | 15-01-2024 |

Only **one record** is affected. Order O102 belonging to Customer C002 has no value recorded in the Sales\_Amount field, making it an incomplete record.

#### b) Why Loading Records with Missing Values is Risky

**Risk 1: Incorrect Aggregations and Revenue Calculations** Sales\_Amount is a numeric field that is directly used in SUM, AVG, and COUNT calculations on dashboards and reports. When a null value is included in these calculations, most databases either ignore the null silently or treat it as zero depending on the function used. Both outcomes produce incorrect totals. A null ignored in SUM means actual revenue is underreported. A null treated as zero drags down average order value, producing misleading performance metrics.

**Risk 2: NOT NULL Constraint Violation at Destination** If the destination table has a NOT NULL constraint defined on the Sales\_Amount column, the database will immediately reject the record during loading and throw an insertion error. Depending on the pipeline's error handling configuration, this could cause the entire batch load to fail, meaning even valid records do not get loaded until the issue is manually resolved.

**Risk 3: Failure of Downstream Business Rule Validations** Many post-load processes such as fraud detection, discount eligibility checks, and revenue threshold alerts depend on Sales\_Amount having a valid value. A null in this field causes these business logic checks to either skip the record entirely or produce undefined results, both of which compromise the reliability of automated processes built on top of the data.

**Risk 4: Misleading Dashboards and Reports** If the null record is loaded without handling, any dashboard displaying order-level data will show a blank or zero for O102. A business user reviewing this record has no way of knowing whether the transaction genuinely had no value or whether the data simply failed to load correctly. This erodes trust in the reporting system over time.

#### Q4. Data Type Validation

Identify records where Sales\_Amount violates expected data type rules.

- Which record(s) will fail numeric validation?
- What would happen if this dataset is loaded into a SQL table with Sales\_Amount as DECIMAL?

#### Answer:

**Expected Data Type:** The Sales\_Amount column should contain only **numeric values (DECIMAL/INT)** as it represents a monetary transaction amount.

#### **Which Records Violate the Expected Data Type?**

Scanning the dataset, two records in the Sales\_Amount column contain non-numeric or missing values.

| Order_ID | Customer_ID | Sales_Amount   | Violation Type                                |
|----------|-------------|----------------|-----------------------------------------------|
| O102     | C002        | Null           | Missing Value no numeric data present         |
| O104     | C004        | Three Thousand | Text String alphabetic entry in numeric field |

#### **a) Records That Will Fail Numeric Validation**

**Order O104 is the primary data type violation.** The value "Three Thousand" is a plain English text string entered in a field that is strictly expected to hold a decimal number. This is a direct data type mismatch and will fail any numeric validation check applied during transformation or loading.

**Order O102 is a secondary concern.** While Null is not a wrong data type in the strictest sense, it still fails numeric validation in scenarios where the column has a NOT NULL constraint or where downstream calculations require a definite numeric value.

#### **b) What Happens When Loaded into a SQL Table with Sales\_Amount as DECIMAL?**

When the ETL pipeline attempts to load this dataset into a SQL table where Sales\_Amount is defined as DECIMAL, the following sequence of events occurs for each problematic record.

**For Order O104 — "Three Thousand"** The SQL engine attempts to cast the string "Three Thousand" into a DECIMAL value. Since alphabetic text cannot be implicitly converted to a number, the engine throws an **implicit conversion error or a type mismatch error** immediately. Depending on the database system and the pipeline's error handling mode, one of two things happens. Either the entire batch load is aborted and no records are inserted at all, or the single offending record is rejected and skipped while the rest of the batch proceeds. In both cases, the record never makes it into the destination table in its current form.

**For Order O102 — Null** If the Sales\_Amount column in the destination table is defined as DECIMAL NOT NULL, the null value triggers a **NOT NULL constraint violation** and the

record is rejected. If the column allows nulls, the record loads successfully but carries the risk of corrupting aggregations and downstream calculations as discussed in the missing value analysis.

Q5. Date Format Consistency

The Order\_Date column has multiple formats.

- a) List all date formats present in the dataset
- b) Why is this a problem during data loading?

Answer:

Date format inconsistency is one of the most common yet overlooked data quality issues in ETL pipelines. While all values in the Order\_Date column may represent valid calendar dates, the manner in which they are written differs across records, and this difference is enough to cause serious problems during data loading.

a) All Date Formats Present in the Dataset

Scanning the Order\_Date column across all six records reveals the following formats.

| Order_ID | Order_Date Value | Format Identified |
|----------|------------------|-------------------|
| O101     | 12-01-2024       | DD-MM-YYYY        |
| O102     | 15-01-2024       | DD-MM-YYYY        |
| O103     | 2024/01/18       | YYYY/MM/DD        |
| O101     | 12-01-2024       | DD-MM-YYYY        |
| O104     | 20-01-2024       | DD-MM-YYYY        |
| O105     | 25-01-2024       | DD-MM-YYYY        |

Two distinct date formats exist in the dataset.

**Format 1 — DD-MM-YYYY** is used by the majority of records including O101, O102, O104, and O105, where the day comes first, followed by month, followed by the four-digit year, all separated by hyphens.

**Format 2 — YYYY/MM/DD** is used exclusively by record O103, where the four-digit year comes first, followed by month, followed by day, all separated by forward slashes.

b) Why This is a Problem During Data Loading

**Problem 1: SQL Destination Table Accepts Only One Defined Format** When a SQL table is created with an Order\_Date column of type DATE or DATETIME, the database engine

expects all incoming values to follow a single consistent format that matches the column definition. When the ETL loader processes the dataset and encounters both DD-MM-YYYY and YYYY/MM/DD formats simultaneously, it can only correctly parse one of them. The record carrying the non-standard format, in this case O103, will either throw a **date parse error** or be **rejected entirely** during the load operation.

**Problem 2: Silent Incorrect Date Parsing** A more dangerous outcome occurs when the database does not reject the inconsistent format but instead attempts to interpret it using its default parsing logic. For example, a system configured to read dates as YYYY/MM/DD may interpret the value 12-01-2024 from O101 incorrectly, reading 12 as the year fragment rather than the day. This produces a **wrong date stored silently** with no error raised, meaning the record loads successfully but carries factually incorrect date information that will corrupt all time-based analysis.

**Problem 3: Failure of Date-Based Queries and Filters** Once loaded, Order\_Date is heavily used in filtering, sorting, trend analysis, and time-series reporting. If dates from different records have been parsed using different interpretations due to format inconsistency, queries such as filtering orders between January 1st and January 31st may return incomplete or wrong results. Records may appear out of sequence or fall outside expected date ranges simply because their dates were misread during loading.

**Problem 4: ETL Pipeline Instability** If the pipeline has strict schema validation enabled, a single format mismatch can cause the entire batch load to fail. This means even the correctly formatted records do not get loaded until the inconsistency in O103 is manually corrected, causing data delays and operational disruption.

#### Q6. Load Readiness Decision

Based on the dataset condition:

- a) Should this dataset be loaded directly into the database? (Yes/No)
- b) Justify your answer with at least three reasons

#### Answer:

##### **a) Should This Dataset Be Loaded Directly Into the Database?**

**No.** This dataset is **not ready for direct loading** into the destination database in its current state.

##### **b) Justification**

**Reason 1: Primary Key Violation Exists** Order\_ID is designated as the Primary Key, yet Order\_ID O101 appears twice in the dataset with identical values across all fields. Loading this dataset directly would either cause the database to throw a duplicate key constraint error and abort the entire load job, or if no constraint is enforced, silently insert a duplicate record that inflates order counts and revenue figures. A primary key violation is one of the most

fundamental structural errors a dataset can carry and is alone sufficient to reject the dataset from being loaded without prior deduplication.

**Reason 2: Invalid and Missing Values in a Critical Numeric Column** The Sales\_Amount column contains two problematic records. Order O104 carries the text string "Three Thousand" in a field defined as DECIMAL, which will cause an immediate type conversion error during loading. Order O102 carries a Null value, which will either violate a NOT NULL constraint or silently corrupt revenue aggregations and dashboard totals. Since Sales\_Amount is a core business metric used in nearly every financial calculation, loading these records without resolution would compromise the accuracy of the entire destination dataset.

**Reason 3: Inconsistent Date Format Across Records** The Order\_Date column contains two different formats, DD-MM-YYYY used by the majority of records and YYYY/MM/DD used by Order O103. The destination database expects a single consistent date format. Loading this dataset as-is risks either a parse error on the inconsistent record or, more dangerously, a silent misinterpretation where dates are stored incorrectly without any error being raised. Either outcome breaks the reliability of all date-based queries, filters, and time-series reports built on this table.

**Reason 4: No Record Has Been Validated Against Business Rules** Beyond structural issues, none of the records have been verified against business constraints such as whether Sales\_Amount is greater than zero, whether Order\_Date falls within a valid operational range, or whether Customer\_IDs correspond to existing master data. Loading data that has not passed business rule validation means the destination system receives records that may be structurally acceptable but operationally meaningless or misleading.

## Q7. Pre-Load Validation Checklist

List the exact pre-load validation checks you would perform on this dataset before loading.

### Answer:

A pre-load validation checklist is a structured set of checks applied to a dataset during the transformation stage to ensure it is clean, consistent, and structurally sound before it is allowed to enter the destination database. The following checklist is built specifically around the issues identified in the given Orders dataset.

## Checklist Overview

### Check 1: Primary Key Uniqueness Validation

Verify that every value in the Order\_ID column is unique across the entire dataset. No two records should carry the same Order\_ID. Flag and remove any duplicate entries before loading. In this dataset, Order\_ID O101 appears twice and must be deduplicated, retaining only one instance.

**Rule:** COUNT(Order\_ID) must equal COUNT(DISTINCT Order\_ID)

## **Check 2: Null and Missing Value Validation**

Inspect all columns for null or empty values, with special attention to critical fields that are used in calculations or have NOT NULL constraints at the destination. Every mandatory field must carry a valid non-null value before the record is approved for loading.

**Rule:** Sales\_Amount, Order\_ID, Customer\_ID, and Order\_Date must not be null for any record.

## **Check 3: Data Type Validation**

Verify that every value in each column matches the expected data type defined in the destination schema. Sales\_Amount must contain only numeric values. Order\_Date must contain only valid date values. Any record carrying a text string in a numeric field or an unrecognizable value in a date field must be rejected or corrected before loading.

**Rule:** Sales\_Amount must be castable to DECIMAL. Order\_Date must be castable to DATE.

## **Check 4: Date Format Standardization Check**

Verify that all values in the Order\_Date column follow a single consistent format before loading. The standard target format should be YYYY-MM-DD as it is universally supported across SQL systems. Any record with a differing format must be converted during transformation. In this dataset, O103 carries the format YYYY/MM/DD and must be reformatted before loading.

**Rule:** All Order\_Date values must match the format YYYY-MM-DD.

## **Check 5: Range and Boundary Validation**

Verify that numeric values fall within realistic and acceptable business boundaries. Sales\_Amount must be greater than zero since a completed order cannot have a zero or negative transaction value. Order\_Date must fall within the valid operational date range of the business and must not be a future date unless the business permits pre-bookings.

**Rule:** Sales\_Amount must be greater than 0. Order\_Date must be a valid past or present date.

## **Check 6: Referential Integrity Validation**

Verify that every Customer\_ID present in the Orders dataset exists in the Customers Master table. Any Order record referencing a Customer\_ID that does not exist in the master table is an orphan record and must be quarantined before loading to prevent broken relationships in the destination system.

**Rule:** Every Customer\_ID in Orders must have a matching entry in Customers\_Master.



## Check 7: Duplicate Record Validation

Beyond primary key checking, perform a full-row duplicate check to identify records that are completely identical across all fields. Even if no primary key constraint exists, full row duplicates corrupt aggregations and must be removed before loading.

**Rule:** No two rows should be identical across all columns simultaneously.

## Check 8: Column Count and Schema Alignment Check

Verify that the incoming dataset contains the exact number of columns expected by the destination table and that each column maps to the correct field in the correct order. A mismatch in column count or column order will cause the loader to insert values into wrong fields or fail the load entirely.

**Rule:** Dataset must contain exactly four columns — Order\_ID, Customer\_ID, Sales\_Amount, Order\_Date — in the correct order.

## Complete Checklist Summary Table

| Check No. | Validation Type          | Column Checked           | Expected Condition       | Status on Given Dataset             |
|-----------|--------------------------|--------------------------|--------------------------|-------------------------------------|
| 1         | Primary Key Uniqueness   | Order_ID                 | All values unique        | <b>Fail</b> — O101 duplicated       |
| 2         | Null / Missing Value     | Sales_Amount             | No nulls allowed         | <b>Fail</b> — O102 is null          |
| 3         | Data Type Validation     | Sales_Amount             | Must be DECIMAL          | <b>Fail</b> — O104 is text          |
| 4         | Date Format Consistency  | Order_Date               | Must be YYYY-MM-DD       | <b>Fail</b> — O103 inconsistent     |
| 5         | Range / Boundary Check   | Sales_Amount, Order_Date | Amount > 0, valid date   | <b>Fail</b> — O102 null amount      |
| 6         | Referential Integrity    | Customer_ID              | Must exist in master     | Needs master table check            |
| 7         | Full Row Duplicate Check | All Columns              | No identical rows        | <b>Fail</b> — O101 fully duplicated |
| 8         | Schema Alignment         | All Columns              | 4 columns, correct order | Pass                                |

## Final Decision Based on Checklist

Out of eight validation checks performed on this dataset, **at least five checks fail**. This confirms that the dataset must be sent back to the transformation stage for correction and must not be loaded into the destination database in its current state. Loading can only proceed once all checks in this checklist return a passing status.

Q8. Cleaning Strategy Describe the step-by-step cleaning actions required to make this dataset load-ready.

### Answer:

#### Step-by-Step Cleaning Strategy to Make the Dataset Load-Ready

A cleaning strategy defines the exact sequence of actions applied during the transformation stage to resolve all identified data quality issues before the dataset is allowed to enter the destination database. The following steps are designed specifically for the given Orders dataset.

##### Step 1: Remove Exact Duplicate Records

The first and most critical action is deduplication. Order\_ID O101 appears twice in the dataset with identical values across all four fields, making it an exact duplicate that violates the Primary Key constraint.

**Action:** Partition records by Order\_ID and apply a row-number function to rank each occurrence. Retain only the first occurrence and permanently drop all subsequent duplicates from the dataset before proceeding further.

**Result:** Dataset reduces from 6 rows to 5 rows. One clean instance of O101 is retained.

##### Step 2: Handle Missing Value in Sales\_Amount

Order O102 carries a Null in the Sales\_Amount column. Since Sales\_Amount is a core financial field used in revenue calculations and subject to NOT NULL constraints at the destination, this must be resolved before loading.

**Action:** First attempt to retrieve the correct value from the source system or transaction logs. If the actual amount is recoverable, replace the null with the confirmed value. If recovery is not possible, route O102 to a quarantine table with a reason code stating "Missing Sales Amount" and exclude it from the main load batch. Never substitute null with zero unless the business explicitly confirms the transaction carried no monetary value.

**Result:** O102 is either corrected and included or safely quarantined and excluded from loading.

### Step 3: Correct Invalid Data Type in Sales\_Amount

Order O104 contains the text string "Three Thousand" in the Sales\_Amount field. This is a direct data type violation since the destination column expects a DECIMAL numeric value.

**Action:** If the intended value is unambiguous and confirmed by the business or source system, convert "Three Thousand" to its numeric equivalent 3000 during the transformation step. If there is any doubt about the correct amount, quarantine the record and flag it for manual review. Never attempt to load a text string directly into a DECIMAL column as it will cause an immediate type conversion error and may abort the entire batch.

**Result:** O104 carries Sales\_Amount as numeric 3000 and proceeds to loading, or is quarantined pending business confirmation.

### Step 4: Standardize Date Format in Order\_Date

The Order\_Date column contains two different formats. Records O101, O102, O104, and O105 use DD-MM-YYYY while O103 uses YYYY/MM/DD. The destination schema requires a single uniform date format.

**Action:** Apply a date parsing and conversion function during transformation that reads each Order\_Date value regardless of its incoming format, correctly interprets the day, month, and year components, and converts every value to the standard YYYY-MM-DD format. For O103, the value 2024/01/18 is correctly interpreted as January 18th 2024 and stored as 2024-01-18. All other records are similarly converted.

**Result:** All records carry Order\_Date in YYYY-MM-DD format, fully consistent with the destination schema.

### Step 5: Perform Data Type Cast Validation

After resolving null and type violations, explicitly validate that every remaining value in every column can be safely cast to its destination data type before triggering the load.

**Action:** Attempt an explicit CAST of each Sales\_Amount to DECIMAL and each Order\_Date to DATE. Any record that fails this cast is immediately flagged and routed to the error log. Records that pass are approved to move forward in the pipeline.

**Result:** All remaining records are confirmed type-compatible with the destination table schema.

### Step 6: Apply Business Rule Validation

With all structural issues resolved, apply domain-specific business logic to confirm that values are not just technically valid but also operationally correct and meaningful.

**Action:** Verify that every Sales\_Amount is greater than zero since a completed order cannot have a zero or negative value. Verify that every Order\_Date is not a future date beyond the

current processing date. Verify that every Customer\_ID has a corresponding entry in the Customers Master table to ensure referential integrity. Any record failing these checks is quarantined with a descriptive error code.

**Result:** Only records satisfying both structural and business rule requirements are cleared for loading.

### Step 7: Final Pre-Load Verification

Before the load is triggered, perform one last verification pass over the fully cleaned dataset to confirm that every identified issue has been resolved and no new issues have been introduced.

**Action:** Run a COUNT vs DISTINCT COUNT check on Order\_ID to confirm zero duplicates remain. Run null checks across all mandatory columns. Confirm all Order\_Date values are in YYYY-MM-DD format. Confirm all Sales\_Amount values are numeric and positive. Reconcile the final record count against the expected count after quarantining.

**Result:** Dataset receives load-ready status and is approved to proceed to the destination database.

.

### Q9. Loading Strategy Selection

Assume this dataset represents daily sales data.

a) Should a Full Load or Incremental Load be used?

b) Justify your choice.

#### Answer:

#### **a) Recommended Strategy: Incremental Load**

For daily sales data, an **Incremental Load** strategy should be used rather than a Full Load.

#### **b) Justification**

**Reason 1: Data Arrives Daily and Grows Continuously** Sales transactions are generated every single day and the dataset grows in size over time. A Full Load would reload the entire historical dataset from scratch every day, meaning thousands or millions of previously loaded and validated records would be reprocessed unnecessarily with every pipeline run.

Incremental Load solves this by extracting and loading only the new or changed records since the last successful pipeline execution, making each daily run significantly faster and more resource efficient.

**Reason 2: Avoiding Redundant Processing of Historical Data** Once a sales transaction has been validated, cleaned, and loaded into the destination database, it does not need to be touched again unless it is corrected or updated. A Full Load ignores this reality and treats every record, old and new, as if it needs to be reloaded every time. Incremental Load respects the fact that historical data is already present and trusted in the destination, and focuses

pipeline effort only on what is genuinely new, which in the case of daily sales data is the previous day's transactions.

**Reason 3: Reduced Risk of Data Corruption Through Overwrites** A Full Load typically truncates or replaces the destination table before inserting fresh data. For daily sales data, this means previously loaded records are deleted and rewritten in every run. If the pipeline fails midway through a Full Load, the destination table may be left empty or partially filled, causing a complete data outage for all downstream dashboards and reports. Incremental Load appends or updates only new records, leaving existing data untouched, which means a pipeline failure on any given day does not affect the integrity of previously loaded historical data.

**Reason 4: Order\_Date Provides a Natural Watermark for Incremental Extraction** The dataset contains an Order\_Date column which serves as a reliable watermark field for implementing incremental logic. The pipeline can store the maximum Order\_Date from the last successful run and in every subsequent execution extract only those records where Order\_Date is greater than that stored watermark. This makes the incremental strategy straightforward to implement on this specific dataset without requiring any complex change detection mechanism.

**Reason 5: Performance and Cost Efficiency** In real-world environments, daily sales datasets for even medium-sized businesses can contain millions of rows accumulated over months and years. Running a Full Load on such a dataset every day consumes large amounts of compute, memory, and network bandwidth. Incremental Load limits each daily run to only the new records generated in the last 24 hours, dramatically reducing processing time, infrastructure cost, and pipeline execution risk.

#### Q10. BI Impact Scenario

Assume this dataset was loaded without cleaning and connected to a BI dashboard.

- a) What incorrect results might appear in Total Sales KPI?
- b) Which records specifically would cause misleading insights?
- c) Why would BI tools not detect these issues automatically?

#### Answer:

##### **a) Incorrect Results in Total Sales KPI**

The Total Sales KPI is the most critical metric on any sales dashboard as it directly represents business revenue. If the uncleaned dataset is loaded and connected to a BI tool, the following incorrect results would appear.

**Inflated Revenue Figure** The duplicate record for Order O101 carries a Sales\_Amount of 4500. Since this record exists twice in the dataset, the Total Sales KPI would count 4500 twice, adding an extra 4500 to the final sum that does not represent any real transaction. The dashboard would display a revenue figure that is higher than actual earnings, giving leadership a false impression of business performance.

**Understated or Incomplete Revenue Figure** Order O102 carries a Null in Sales\_Amount. Most BI aggregation functions such as SUM ignore null values during calculation. This means the transaction from O102 contributes nothing to the Total Sales KPI even though it represents a real order that likely had a monetary value. The result is a revenue figure that is lower than the true total, making business performance appear worse than it actually is.

**Unpredictable Aggregation Due to Text Value** Order O104 contains "Three Thousand" as its Sales\_Amount. When a BI tool attempts to aggregate this column, it may either throw a rendering error, display the KPI as blank or broken for that record, or in some tools attempt to cast the text and silently assign it a zero value. In any case the true value of 3000 is excluded from the Total Sales KPI, further understating actual revenue.

Combining all three effects, the Total Sales KPI on the dashboard would simultaneously overcount due to duplication and undercount due to nulls and invalid types, producing a figure that is wrong in multiple directions and carries no reliable business meaning.

## **b) Records That Would Cause Misleading Insights**

**Order O101 — Duplicate Record** This record appears twice and causes the Total Sales KPI to be inflated by 4500. Any metric built on order counts such as average order value, transactions per day, or customer purchase frequency would also be distorted because the same transaction is counted as two separate events. Customer C001 would appear to have made two purchases when in reality only one occurred, skewing customer behavior analysis and loyalty metrics.

**Order O102 — Null Sales Amount** This record causes the Total Sales KPI to be understated. Beyond the KPI itself, any trend analysis comparing daily revenue across dates would show January 15th 2024 as a lower performing day than it actually was, because the transaction for that date contributes zero to the sum. Average order value calculations would also be affected depending on whether the BI tool counts O102 as a valid order in the denominator while excluding its null amount from the numerator.

**Order O104 — Text Value in Sales Amount** This record causes a data type conflict in the aggregation. The value "Three Thousand" is excluded from all numeric calculations, meaning 3000 in genuine revenue is invisibly missing from every Sales KPI, trend line, and product performance report. If the BI tool displays an error indicator for this record, it may also trigger visual anomalies on the dashboard that confuse business users without clearly explaining the root cause.

**Order O103 — Inconsistent Date Format** This record carries the date in YYYY/MM/DD format while all others use DD-MM-YYYY. Depending on how the BI tool parses this field, January 18th 2024 may be misread as a completely different date or displayed as an invalid entry. This would cause O103 to either disappear from date-filtered views, appear under a wrong time period, or break the continuity of time-series trend charts, making the sales trend for January appear incomplete or incorrectly distributed across dates.

### **c) Why BI Tools Would Not Detect These Issues Automatically**

**BI Tools Are Visualization Engines, Not Validation Engines** Tools such as Power BI, Tableau, and Looker are designed to read data from a connected source and render it visually as charts, KPIs, and tables. They do not inspect the quality or logical correctness of the data they receive. They trust the source completely and display whatever values are present, whether those values are accurate, duplicated, or meaningless. Data quality enforcement is the responsibility of the ETL pipeline, not the BI layer.

**Duplicates Are Invisible to Aggregation Functions** When a BI tool computes Total Sales using a SUM function, it adds every value in the Sales\_Amount column including duplicate rows. It has no built-in awareness that two rows represent the same real-world transaction. From the tool's perspective, six rows with valid amounts simply means six amounts to add together. There is no native duplication alert in standard BI aggregation logic.

**Null Handling Is Silent and Automatic** Most BI tools handle null values by excluding them from numeric aggregations without raising any visible warning to the end user. The dashboard simply shows a total that quietly omits the null record. A business user looking at the Total Sales KPI has no indication that a transaction was excluded from the calculation, making the understatement invisible and therefore unchallenged.

**Type Conflicts Are Absorbed or Hidden** When a BI tool encounters a text value in a numeric column, it typically either excludes the record silently, renders a blank cell, or in some cases displays a generic data error icon at the field level. None of these behaviors communicate to the business user that a real transaction worth 3000 is missing from the total. The dashboard continues to function and look normal, which is precisely what makes this type of error so dangerous.

**Date Misinterpretations Produce No Visual Alarm** If a date is parsed incorrectly due to format inconsistency, the BI tool stores and displays the misread value as if it were correct. A sales record that belongs to January 18th may appear under a completely different month or year, and the trend chart for January would simply appear to have one fewer transaction without any error message or flag being raised.